COMPUTATIONAL METHOD

LAB FILE



MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY

C4 , Janakpuri, New Delhi – 110058

Electronics and Communication Engineering Department

Name:- Ronit Chauhan

Class Serial No.:- 16

Enrollment No.:- 02315002825

Class:- ECE — 1

Submitted To:

Dr. Geetanjali

Dr. Deeba Naqvi

INDEX

Exp no.	Date	Exp name	Page no.	Teacher's sign	Remarks

Practical : 1 (a)

Linear equation in 1 variable

Aim : Program to find value of x in Linear equation.

Code :

```
#include <stdio.h>
int main() {
   float a, b, x;
   printf("Equation: ax + b = 0\n");
   printf("Enter a and b: ");
   scanf("%f %f", &a, &b);
   if (a == 0) {
      printf("Not a linear equation (a=0).\n");
   } else {
      x = -b / a;
      printf("Solution: x = %.2f\n", x);
   }
   return 0;
}
```

Output:

```
linear equation : ax + b = 0
Enter a and b = 5 3
Solution: x = -0.600000
```

Practical : 1 (b)

Linear equation in 2 variable

Aim : Program to find value of x,y in Linear equation.

Code:

```
#include <stdio.h>
int main() {
   float a1, b1, c1, a2, b2, c2, x, y, eq1,eq2;
   printf("Equations: a1x + b1y = c1 & a2x + b2y = c2\n");
   printf("Enter a1, b1, c1: ");
   scanf("%f %f %f", &a1, &b1, &c1);
   printf("Enter a2, b2, c2: ");
   scanf("%f %f %f", &a2, &b2, &c2);
   //By applying elimination rule
   float denominator = (b1 * a2 - b2 * a1);

   if (denominator == 0) {
   printf("No unique solution (lines are parallel or coincident).\n");
}
   else {
   y = (c1 * a2 - c2 * a1) / denominator;
     if (a1 != 0) {
      x = (c1 - b1 * y) / a1;
     } else {
      x = (c2 - b2 * y) / a2;
     }

   printf("Solutions are x = %.2f, y = %.2f\n", x, y);
}
   return 0;
}
```

Output:

```
Equations: a1x + b1y = c1 & a2x + b2y = c2
Enter a1, b1, c1: 11
22
33
Enter a2, b2, c2: 33
22
11
Solutions are x = -1.00, y = 2.00
```

Practical : 1 (c)

Quadratic Equation

Aim : Program to find value of x in Quadratic equation.

Code:

```
#include <stdio.h>
#include <math.h>
int main() {
   float a, b, c, discriminant, x1, x2, realPart, imagPart;
   printf("Quadratic equation : ax^2 + bx + c = 0\n");
   printf("Enter value of a, b, c: ");
   scanf("%f %f %f", &a, &b, &c);
   discriminant = b * b - 4 * a * c;
   if (discriminant > 0) {
      x1 = (-b + sqrt(discriminant)) / (2 * a);
      x2 = (-b - sqrt(discriminant)) / (2 * a);
      printf("Roots are real and distinct: %.2f and %.2f\n", x1, x2);
   } else if (discriminant == 0) {
      x1 = -b / (2 * a);
      x2 = x1;
      printf("Roots are real and equal: %.2f and %.2f\n", x2, x1);
   } else {
      realPart = -b / (2 * a);
      imagPart = sqrt(-discriminant) / (2 * a);
      printf("Roots are complex: %.2f + %.2fi and %.2f - %.2fi\n", realPart, imagPart,
realPart, imagPart);
   }
   return 0;
}
```

Output:

```
Quadratic equation : ax^2 + bx + c = 0
Enter value of a, b, c: 22
32
44
Roots are complex: -0.73 + 1.21i and -0.73 - 1.21i
```

Practical : 2

Bisection Method

Aim : Program for finding roots of equation x^3-x-5=0 using Bisection method.

Code:

```
#include <stdio.h>
#include <math.h>
double f(double x) {
  return x*x*x - x - 5;
}
int main() {
  double a = 2, b = 3, c, e = 0.000001;
  if (f(a) * f(b) > 0) {     // checking if intervals are valid or not
    printf("Invalid interval\n");
    return 0;
  }
  printf("Bisection Method for x^3 - x - 5 = 0\n");
  int iter = 1;
  while ((b - a) >= e){     // works only when value is greater than 0,0000001
    c = (a + b) / 2; // mid point
    printf("Iteration %d: x = %f\n", iter++, c);
    if (f(a) * f(c) < 0){     // if answer is positive sooo
      b = c;
    }
    else{         // if answer is negative soooo
      a = c;
    }
  };
  printf("Root is approximately: %f\n", c);
  return 0;
}
```

Output:

```
Bisection Method for x^3 - x - 5 = 0
Iteration 1: x = 5.500000
Iteration 2: x = 3.250000
Iteration 3: x = 2.125000
Iteration 4: x = 1.562500
Iteration 5: x = 1.843750
Iteration 6: x = 1.984375
Iteration 7: x = 1.914063
Iteration 8: x = 1.878906
Iteration 9: x = 1.896484
Iteration 10: x = 1.905273
Iteration 11: x = 1.900879
Iteration 12: x = 1.903076
Iteration 13: x = 1.904175
Iteration 14: x = 1.903625
Iteration 15: x = 1.903900
Iteration 16: x = 1.904037
Iteration 17: x = 1.904106
Iteration 18: x = 1.904140
Iteration 19: x = 1.904158
Iteration 20: x = 1.904166
Iteration 21: x = 1.904162
Iteration 22: x = 1.904160
Iteration 23: x = 1.904161
Iteration 24: x = 1.904161
Root is approximately: 1.904161
```

Practical : 3

Secant Method

Aim : Program for finding roots of of equation x^3-x-5=0 using Secant method.

Code:

```
#include <stdio.h>
#include <math.h>
double f(double x) {
   return x*x*x - x - 5;
}
int main() {
   double x0 , x1 , x2 ,e, iter;
   x0 = 1;
   x1 = 2;
   e = 0.000001;
   iter = 1;
   while (fabs(x1-x0) >= e)
   {  x2 = x1 - ((f(x1)*(x1-x0))/(f(x1) - f(x0)));
      x0 = x1;
      x1 = x2;
      iter++;
      printf("Iteration :%.6f | roots:x0 : %.6f x1 : %.6f \n",iter,x0,x1);
   }
   printf("roots:%.6f\n",x2);
   return 0;
}
```

Output:

```
Iteration :2.000000 | roots:x0 : 2.000000 x1 : 1.833333
Iteration :3.000000 | roots:x0 : 1.833333 x1 : 1.900277
Iteration :4.000000 | roots:x0 : 1.900277 x1 : 1.904325
Iteration :5.000000 | roots:x0 : 1.904325 x1 : 1.904160
Iteration :6.000000 | roots:x0 : 1.904160 x1 : 1.904161
roots:1.904161
```

Practical : 4

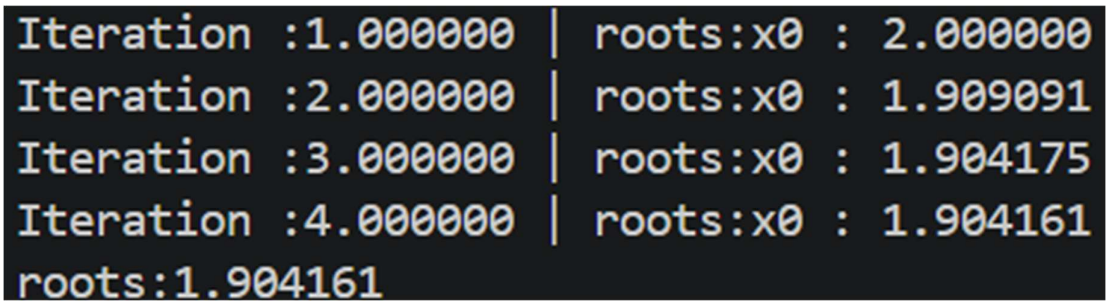
Newton Raphson Method

Aim: Program for finding roots of of equation $x^3-x-5=0$ using Newton Raphson method.

Code:

```
#include <stdio.h>
#include <math.h>
double f(double x) {
   return x*x*x - x - 5;
}
double df(double x){
   return 3*x*x - 1;
}
int main() {
   double x0 , x1 ,e, iter;
   x0 = 1;
   x1 = 2;
   e = 0.000001;
   iter = 1;
   while (fabs(x1-x0) >= e)
   {
      x0 = x1;
      x1 = x0 - f(x0)/df(x0);
      printf("Iteration :%.6f | roots:x0 : %.6f \n" , iter , x0);
      iter++;
   }
   printf("roots:%.6f\n",x1);
   return 0;
}
```

Output:


Iteration :1.000000 | roots:x0 : 2.000000
Iteration :2.000000 | roots:x0 : 1.909091
Iteration :3.000000 | roots:x0 : 1.904175
Iteration :4.000000 | roots:x0 : 1.904161
roots:1.904161

Practical : 5

Matrix Operations

Aim: Program to perform basic Matrix Operations.

- Addition
- Subtraction
- Multiplication
- Inverse
- Adjoint
- Determinant
- Eigenvalues
- Simultaneous Equations
- Orthogonal Check

Code:

```
#include <math.h>
#include <stdio.h>
#define Max 10
float determinant_calc(float a[Max][Max], int n) {
 if (n == 1) return a[0][0];
 if (n == 2) return a[0][0] * a[1][1] - a[0][1] * a[1][0];
 float det = 0, sub[Max][Max];
 int sign = 1;
 for (int x = 0; x < n; x++) {
  int subi = 0;
  for (int i = 1; i < n; i++) {
   int subj = 0;
   for (int j = 0; j < n; j++) {
    if (j == x) continue;
    sub[subi][subj++] = a[i][j];}
   subi++;}
  det += sign * a[0][x] * determinant_calc(sub, n - 1);
  sign = -sign;}
 return det;
}

void getCofactor(float a[Max][Max], float temp[Max][Max], int p, int q, int n) {
 int i = 0, j = 0;
 for (int row = 0; row < n; row++) {
  for (int col = 0; col < n; col++) {
   if (row == p || col == q) continue;
   temp[i][j++] = a[row][col];
   if (j == n - 1) { j = 0; i++; }}}}

void adjoint_calc(float a[Max][Max], float adj[Max][Max], int n) {
 if (n == 1) { adj[0][0] = 1; return; }
```

```
  int sign = 1; float temp[Max][Max];
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      getCofactor(a, temp, i, j, n);
      sign = ((i + j) % 2 == 0) ? 1 : -1;
      adj[j][i] = (sign) * (determinant_calc(temp, n - 1));
    }
  }
}

void input_matrix(float a[Max][Max], int* r, int* c, int is_square) {
  printf("Enter rows and columns: ");
  if (is_square) {
    if (scanf("%d", r) != 1) return;
    *c = *r;
  } else {
    if (scanf("%d %d", r, c) != 2) return;
  }
  printf("Enter elements: ");
  for (int i = 0; i < *r; i++)
    for (int j = 0; j < *c; j++)
      if (scanf("%f", &a[i][j]) != 1) return;
}
void addition() {
  float a[Max][Max], b[Max][Max]; int r, c;
  printf("Enter rows and columns: ");
  if (scanf("%d %d", &r, &c) != 2) return;
  printf("Enter first matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
      if (scanf("%f", &a[i][j]) != 1) return;
  printf("Enter second matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
      if (scanf("%f", &b[i][j]) != 1) return;
  printf("Sum:\n");
  for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) printf("%.2f ", a[i][j] + b[i][j]);
    printf("\n");
  }
}
```

```
void subtraction() {
  float a[Max][Max], b[Max][Max]; int r, c;
  printf("Enter rows and columns: ");
  if (scanf("%d %d", &r, &c) != 2) return;
  printf("Enter first matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
      if (scanf("%f", &a[i][j]) != 1) return;
  printf("Enter second matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
      if (scanf("%f", &b[i][j]) != 1) return;
  printf("Difference:\n");
  for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) printf("%.2f ", a[i][j] - b[i][j]);
    printf("\n");
  }
}

void multiplication() {
  float a[Max][Max], b[Max][Max], mul[Max][Max] = { 0 }; int r1, c1, r2, c2;
  printf("Enter first matrix\n");
  input_matrix(a, &r1, &c1, 0);
  printf("Enter second matrix\n");
  input_matrix(b, &r2, &c2, 0);
  if (c1 != r2) {
    printf("Cannot multiply.\n");
    return;
  }
  printf("Product:\n");
  for (int i = 0; i < r1; i++) {
    for (int j = 0; j < c2; j++) {
      for (int k = 0; k < c1; k++) mul[i][j] += a[i][k] * b[k][j];
      printf("%.2f ", mul[i][j]);
    }
    printf("\n");
  }
}

void inverse() {
```

```
 float a[Max][Max], adj[Max][Max]; int n;
 printf("Enter the matrix\n");
 input_matrix(a, &n, &n, 1);
 float det = determinant_calc(a, n);
 if (det == 0) printf("Inverse does not exist.\n");
 else {
  adjoint_calc(a, adj, n); printf("Inverse:\n");
  for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) printf("%.4f ", adj[i][j] / det);
   printf("\n");
  }
 }
}

void adjoint_op() {
 float a[Max][Max], adj[Max][Max]; int n;
 printf("Enter the matrix\n");
 input_matrix(a, &n, &n, 1); adjoint_calc(a, adj, n);
 printf("Adjoint:\n");
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) printf("%.4f ", adj[i][j]);
  printf("\n");
 }
}

void determinant() {
 float a[Max][Max]; int n;
 printf("Enter the matrix\n");
 input_matrix(a, &n, &n, 1);
 printf("Determinant = %.4f\n", determinant_calc(a, n));
}

void eigenvalue() {
 float a[Max][Max], x[Max], nx[Max]; int n;
 printf("Enter the matrix\n");
 input_matrix(a, &n, &n, 1);
 printf("Enter initial guess vector: ");
 for (int i = 0; i < n; i++)
  if (scanf("%f", &x[i]) != 1)
   return;
 float lambda = 0, lambda_old = 0, max_val; int iter = 0;
```

```
  do {
    lambda_old = lambda;
    for (int i = 0; i < n; i++) {
      nx[i] = 0; for (int j = 0; j < n; j++) nx[i] += a[i][j] * x[j];
    }
    max_val = fabs(nx[0]);
    for (int i = 1; i < n; i++) if (fabs(nx[i]) > max_val) max_val = fabs(nx[i]);
    lambda = max_val;
    for (int i = 0; i < n; i++) x[i] = nx[i] / lambda;
    iter++;
  } while (fabs(lambda - lambda_old) > 0.0001 && iter < 1000);
  printf("Dominant Eigenvalue: %.4f\n", lambda);
}

void simultaneous_equations() {
  float a[Max][Max + 1]; int n;
  printf("Enter number of variables: ");
  if (scanf("%d", &n) != 1)
    return;
  printf("Enter augmented matrix: ");
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n + 1; j++)
      if (scanf("%f", &a[i][j]) != 1)
        return;
  for (int i = 0; i < n; i++) {
    if (a[i][i] == 0.0)
      return;
    for (int j = 0; j < n; j++) {
      if (i != j) {
        float ratio = a[j][i] / a[i][i];
        for (int k = 0; k < n + 1; k++) a[j][k] = a[j][k] - ratio * a[i][k];
      }
    }
  }
  printf("Solutions:\n");
  for (int i = 0; i < n; i++) printf("x%d = %.4f\n", i + 1, a[i][n] / a[i][i]);
}

void orthogonal() {
  float a[Max][Max], mul[Max][Max] = { 0 }; int n, isOrtho = 1;
  printf("Enter the matrix\n");
```

```
 input_matrix(a, &n, &n, 1);
 for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
   for (int k = 0; k < n; k++) mul[i][j] += a[i][k] * a[j][k];
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   if (i == j && (mul[i][j] < 0.999 || mul[i][j] > 1.001)) isOrtho = 0;
   if (i != j && (mul[i][j] < -0.001 || mul[i][j] > 0.001)) isOrtho = 0;
  }
 }
 printf(isOrtho ? "Matrix is Orthogonal.\n" : "Matrix is NOT Orthogonal.\n");
}

int main() {
 int choice;
 do {
  printf("\n==== Matrix Operations Combined ====\n");
  printf("1. Addition\n2. Subtraction\n3. Multiplication\n4. Inverse\n5. Adjoint\n6.
Determinant\n7. Eigenvalues\n8. Simultaneous Equations\n9. Orthogonal Check\n0.
Exit\nEnter your choice: ");
  if (scanf("%d", &choice) != 1) break;
  if (choice == 1) addition();
  else if (choice == 2) subtraction();
  else if (choice == 3) multiplication();
  else if (choice == 4) inverse();
  else if (choice == 5) adjoint_op();
  else if (choice == 6) determinant();
  else if (choice == 7) eigenvalue();
  else if (choice == 8) simultaneous_equations();
  else if (choice == 9) orthogonal();
  else if (choice == 0) printf("Exiting...\n");
  else printf("Invalid choice!\n");
 } while (choice != 0);
 return 0;
}
```

Output:

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 1
Enter rows and columns: 2 2
Enter first matrix
Enter elements: 1 2 3 4
Enter second matrix
Enter elements: 3 4 5 6
Sum:
4.00 6.00
8.00 10.00
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 2
Enter rows and columns: 2 2
Enter first matrix
Enter elements: 1 2 3 4
Enter second matrix
Enter elements: 3 4 5 6
Difference:
-2.00 -2.00
-2.00 -2.00
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 3
Enter first matrix
Enter rows and columns: 2 3
Enter elements: 1 2 3 4 5 6
Enter second matrix
Enter rows and columns: 3 2
Enter elements: 7 8 9 10 11 12
Product:
58.00 64.00
139.00 154.00
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 4
Enter the matrix
Enter rows and columns: 2
Enter elements: 4 5 6 7
Inverse:
-3.5000 2.5000
3.0000 -2.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 5
Enter the matrix
Enter rows and columns: 2
Enter elements: 1 2 3 4
Adjoint:
4.0000 -2.0000
-3.0000 1.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 6
Enter the matrix
Enter rows and columns: 3
Enter elements: 1 2 3 4 5 6 7 8 9
Determinant = 0.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 7
Enter the matrix
Enter rows and columns: 2
Enter elements: 1 2 3 4
Enter initial guess vector: -1 1
Dominant Eigenvalue: 5.3723
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 8
Enter number of variables: 2
Enter augmented matrix: 2 1 5 1 -1 1
Solutions:
x1 = 2.0000
x2 = 1.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 9
Enter the matrix
Enter rows and columns: 2
Enter elements: 1 0 0 1
Matrix is Orthogonal.
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 0
Exiting...
```

Practical : 6

Lagrange's Interpolation

Aim: Program to implement Langrange's Interpolation formula.

Code:

```
#include <stdio.h>
#include <stdlib.h>

double lagrange_interpolation(double* x_vals, double* y_vals, int n, double target_x) {
 double interpolated_y = 0.0;

 for (int i = 0; i < n; i++) {
  double basis_polynomial = y_vals[i];
  for (int j = 0; j < n; j++) {
   if (i != j) {
    basis_polynomial *= (target_x - x_vals[j]) / (x_vals[i] - x_vals[j]);
   }
  }
  interpolated_y += basis_polynomial;}
 return interpolated_y;
}

int main() {
 int n;
 double target_x;

 printf("Enter the number of data points (n): ");
 if (scanf("%d", &n) != 1 || n <= 0) {
  printf("Error: Number of points must be a positive integer.\n");
  return 1;}

 double* x_vals = (double*)malloc(n * sizeof(double));
 double* y_vals = (double*)malloc(n * sizeof(double));

 if (x_vals == NULL || y_vals == NULL) {
  printf("Error: Memory allocation failed!\n");
  return 1;}

 printf("\nEnter the data points (x and corresponding y):\n");
```

```
 for (int i = 0; i < n; i++) {
  printf("x[%d] = ", i);
  scanf("%lf", &x_vals[i]);
  printf("y[%d] = ", i);
  scanf("%lf", &y_vals[i]);
 }

 printf("\nEnter the value of x to interpolate: ");
 scanf("%lf", &target_x);

 double result = lagrange_interpolation(x_vals, y_vals, n, target_x);

 printf("\n--------------------------------------------------\n");
 printf("The estimated value of y at x = %.4f is %.4f\n", target_x, result);
 printf("--------------------------------------------------\n");

 free(x_vals);
 free(y_vals);

 return 0;
}
```

Output:

```
Enter the number of data points (n): 6

Enter the data points (x and corresponding y):
x[0] = 1
y[0] = 2
x[1] = 2
y[1] = 3
x[2] = 3
y[2] = 4
x[3] = 4
y[3] = 5
x[4] = 5
y[4] = 6
x[5] = 6
y[5] = 7

Enter the value of x to interpolate: 10

--------------------------------------------------
The estimated value of y at x = 10.0000 is 11.0000
--------------------------------------------------
```